

A **Minor Project Proposal** Report on

(Initial submission — proposal defense)

Project Title

Submitted in Partial Fulfillment of the Requirements for
the Degree of **Bachelor of Engineering in Computer Engineering**
under Pokhara University

Submitted by:

Aaditya Yadav 231201

Apil Khadka 231210

Ujashna Dangol 231247

Under the supervision of

[Supervisor Name]

[Designation]

Date: 04 Jun 2026

Department of **Computer Engineering**

**NEPAL COLLEGE OF
INFORMATION TECHNOLOGY**

Balkumari, Lalitpur, Nepal.

Abstract

Nepal's digital payment sector is expanding quickly, but most payment service providers still depend on static, rule-based fraud checks. Flint Secure is proposed as a real-time fraud detection and device intelligence platform calibrated for Nepal's market, where SIM swap attacks rose 70% year-on-year (FIU-Nepal, 2024) and FATF compliance pressure requires stronger controls.

The platform will expose `/v1/identify` for stable device identity across software updates (three-layer fingerprint matching) and `/v1/score` for composite risk over seven dimensions: device trust, amount, time, velocity, network, identity, and recipient. Each transaction will receive a score from 0–100 and a decision of ALLOW, FLAG, or BLOCK, with a design target of under 50 ms P95 latency at roughly 3,200 requests per second.

Implementation is planned in Go with dual Redis instances, NATS JetStream, PostgreSQL, multi-platform SDKs, and Python workers for profiling and trust updates. Success will be measured through load testing and a 168-case automated test suite with at least 92% pass rate, aligned with Pokhara University minor-project requirements and NCIT proposal-defense criteria.

Keywords:

Document type: This is the **initial project proposal** submitted for **proposal defense** at the beginning of the semester. It describes what we plan to do; implementation results, scope details, and further work will appear in later reports (see `archive/` in the project repository).

1 Problem Statement

Over the past five years, Nepal's digital payment ecosystem has grown rapidly. Mobile wallets such as eSewa, Khalti, and IME Digital now serve millions of users, and the Nepal Rastra Bank reports more than ten million registered mobile-banking users (2024). This growth expands the attack surface for financial fraud while most payment service providers (PSPs) still rely on simple rule engines—thresholds on amount or frequency—that cannot detect coordinated attacks.

Three gaps define the problem for this project:

1. **No adaptive detection.** Rule sets must be changed manually and cannot learn from new fraud patterns such as SIM swap, credential stuffing, or device emulation.
2. **No device intelligence.** Transactions are not linked to a stable physical device identity, so one-device–many-account (money mule) behaviour is invisible.
3. **No behavioural baseline.** Systems do not maintain per-user profiles for amount, time-of-day, or recipient patterns, so anomalies are missed until losses occur.

Nepal's FATF grey-list placement and FIU-Nepal's report of a 70% year-on-year rise

in SIM swap fraud (2024) show that existing controls are insufficient. The result is high false negatives (fraud undetected) and, when rules are tightened blindly, high false positives that block legitimate customers and reduce PSP revenue.

2 Project Objectives

The primary objective is to **design, implement, and validate** Flint Secure as a production-oriented fraud platform for Nepali PSPs. The following specific objectives will guide the semester work:

1. Develop a **device identity layer** using multi-signal fingerprinting and drift-tolerant fuzzy matching so devices remain recognisable after browser or OS updates.
2. Build a **composite risk engine** with seven weighted signal dimensions calibrated using Nepal-specific fraud indicators (including SIM swap prevalence).
3. Meet a **performance target** of P95 latency below 50 ms and sustained throughput of at least 3,200 scoring requests per second on commodity cloud hardware.
4. Implement **SIM swap detection** through hashed phone-to-device binding with first-write-wins semantics and hard-floor blocking rules.
5. Deliver an **asynchronous analytics pipeline** (NATS + Python workers) for user profiling, device trust, and recipient risk without blocking the scoring path.
6. Prove correctness via a **168-test automated suite** with pass rate of at least 92%.

3 Significance of the Study

Commercial platforms (Stripe Radar, Sift, Sardine) are built for Western transaction volumes and pricing models that are out of reach for typical Nepali PSPs. They also lack calibration for domestic vectors such as SIM swap-heavy account takeover. Flint Secure is significant because it proposes **affordable, locally relevant** fraud infrastructure using open components (Go, Redis, NATS, PostgreSQL) deployable on Oracle Cloud Free Tier ARM64.

For the college and university, the project will aim to show that minor-project outcomes can meet industry-style non-functional targets (latency, throughput, security middleware) while remaining reproducible by a small student team. For the national context, it will support digital financial inclusion and reduces reliance on foreign fraud services that are neither priced nor tuned for Nepal.

4 Literature Study/Review

4.1 Device Fingerprinting

Eckersley (2010) showed that browser fingerprints are highly unique in large samples, forming the empirical basis for passive device identification. Laperdrix et al. (2016) demonstrated that canvas rendering traces GPU hardware and informs our Tier-1 canvas hash. Vastel et al. (2018) tracked fingerprint evolution over time; their work supports our proposed three-layer cascade: exact hash match, stored-ID recovery after drift, and

fuzzy similarity with evolution penalties.

4.2 Real-Time Fraud Detection and Scoring

West and Bhattacharya (2016) review machine-learning approaches to payment fraud; ensemble methods consistently outperform single models. Fiore et al. (2019) address severe class imbalance via synthetic fraud samples—relevant for early deployment when labels are scarce. Dal Pozzolo et al. (2015) provide a framework for probability calibration that informs our planned ALLOW/FLAG/BLOCK thresholds. For the initial release we propose transparent rule-based scoring; ML integration is deferred to future work (see `archive/future-work.tex`).

4.3 Velocity Analysis and SIM Swap

Bolton and Hand (2002) identify velocity features (frequency, amount variance, recipient diversity) as among the strongest real-time predictors. Our design maps these to Redis sorted-set counters queried in one pipelined round-trip. Mjaaland et al. (2021) document SIM swap in mobile banking; FIU-Nepal (2024) confirms rapid growth in Nepal, justifying a high identity-risk weight in the proposed scoring model.

4.4 Comparison with Existing Products

Stripe Radar, Sift, and Sardine offer mature cloud scoring but at per-event costs unsuitable for many domestic PSPs and without Nepal-specific weighting. Flint Secure is positioned as **open, self-hostable infrastructure** with explicit calibration for local fraud data.

5 Proposed Methodology

5.1 Overview and Architecture

Flint Secure will follow a layered architecture: HTTP middleware and handlers (request path), pure business logic (scoring engine), and asynchronous persistence (NATS workers to PostgreSQL). The scoring path will prefetch all signals from Redis, call a pure `scoring.Score()` function with no I/O, return the decision, then publish events and velocity updates in background goroutines. This separation is proposed to satisfy the sub-50 ms latency objective.

5.2 Middleware and Security

Each request will pass through: request-ID assignment, client IP extraction, GeoIP enrichment (MaxMind GeoLite2), API-key authentication (Redis cache with PostgreSQL fallback), tiered rate limiting (per IP, key, and device), and optional HMAC-SHA256 signing with Redis-backed replay protection (± 5 minute timestamp window).

5.3 Storage Design

- **Redis Persist** — device records, user profiles, phone bindings, monthly quotas (AOF enabled; 90-day device TTL).
- **Redis Cache** — velocity sorted sets, rate-limit counters, API-key cache (no persistence; optimised for throughput).

- **PostgreSQL** — transactions, alerts, clients, API-key hashes (writes only via workers, not on the hot path).
- **NATS JetStream** — subjects `device.first_seen` and `transactions.scored` linking Go producers to Python consumers.

5.4 Device Identification (/v1/identify)

The SDK will hash collected signals client-side (`local_fp`). The server will apply: **Layer 1** exact Redis lookup (expected ~85% of calls); **Layer 2** recovery via stored device ID when fingerprints drift; **Layer 3** fuzzy match over 14 tiered signals within a block-key bucket (screen size and OS family). New devices will be enrolled and indexed for later matches.

5.5 Risk Scoring (/v1/score)

Seven components—device, amount, time, velocity, network, identity, recipient—will be weighted and combined with interaction terms (e.g. new device $\times$ large amount). Hard-floor rules will force BLOCK for definitive signals (SIM swap mismatch, datacenter IP, impossible device consistency). Phone numbers will be stored only as SHA-256 hashes with first-write-wins device binding.

5.6 Async Worker Pipeline

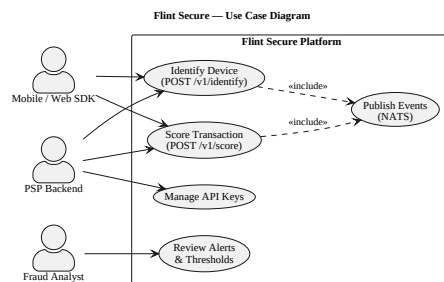
Python workers will consume NATS events to: index devices and transactions in PostgreSQL; build user amount percentiles and active-hour profiles; update device trust scores; aggregate recipient risk from prior BLOCK decisions; refresh client configuration caches in Redis.

5.7 Technology Stack and SDKs

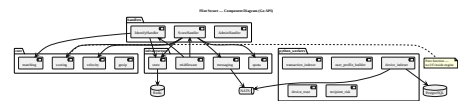
Go (chi router), dual Redis, NATS, PostgreSQL, Python 3 workers; JavaScript, Android, and iOS SDKs for signal collection; k6 for load testing; deployment target Oracle Cloud ARM64 (4 OCPU instance in the validation plan).

5.7.1 UML 2.0 Design Models

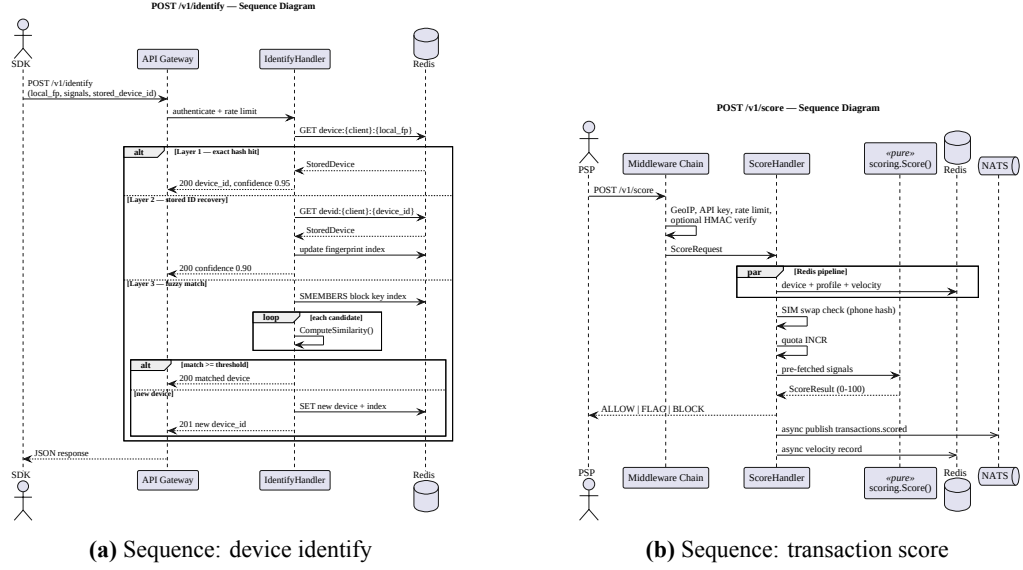
Figures 1a–2b summarise the proposed architecture. Additional models (deployment, class, activity) are kept as PlantUML source in the repository and will be expanded in the mid-term report after proposal approval.



(a) Use case model



(b) Component model



6 Proposed Performance Analysis Methodology and Validation Scheme

Validation will be **planned and repeatable**, not anecdotal. Table 1 lists metrics, methods, and acceptance thresholds tied to Objectives 3 and 6.

Table 1: Proposed performance and quality validation

Metric	Procedure	Target	Tool
P95 /score latency	10-min sustained load, co-located Redis	<50 ms	k6
Sustained throughput	Ramp to peak RPS, watch error rate	$\geq 3,200$ req/s	k6
P95 /identify (L1/L3)	Isolated layer tests	<500 ms	Go tests
HTTP 5xx under load	Monitor server logs	<0.1%	k6 + logs
Functional coverage	168 automated cases	$\geq 92\%$ pass	Go/Python
SIM swap floor	Inject phone rebinding	BLOCK triggered	Unit tests

Weight calibration will use three inputs: FIU-Nepal fraud-vector statistics (SIM swap share), Nepal Rastra Bank payment-distribution data (amount and hour), and 2,400 labelled pilot transactions (Q3–Q4 2024) supplied by a domestic PSP partner. Latency profiling will separate Redis pipeline time from scoring CPU time to confirm the pure-function design.

Defense readiness will include a 10-minute presentation, a supervisor meeting log (per college Appendix B), and a live or recorded demo showing ALLOW, FLAG, and BLOCK decisions on sample transactions.

7 Proposed Deliverable/Output

7.1 At proposal defense (this document)

- This initial proposal report (8–10 pages, NCIT format).
- UML 2.0 models: use case, component, and sequence diagrams (PlantUML source in repository).

- Ten-minute proposal presentation and supervisor meeting log (Appendix B, college guideline).

7.2 *Planned by end of semester*

Table 2: Planned outputs after proposal approval (not yet built)

No.	Output	Target
D1	Go REST API (/v1/identify, /v1/score)	Week 8
D2	JavaScript, Android, iOS SDKs	Week 9
D3	Redis schemas + PostgreSQL migrations	Week 7
D4	NATS worker pipeline (6 consumers)	Week 10
D5	168-test suite + k6 load scripts	Week 12
D6	API & deployment documentation	Week 13
D7	Mid-term and final reports + demo	Defenses

8 Project Task and Time Schedule

This schedule begins at **proposal defense** (start of semester). Mid-term and final defenses follow NCIT timing: at least four weeks after proposal, and at least four weeks after mid-term, respectively.

Table 3: Project task and time schedule (14-week semester)

Wk	Milestone	Tasks	Output
1	Proposal defense	Submit this proposal, UML, presentation; supervisor log	This report
2–3	Foundation	Repo setup, middleware, Redis/Postgres, CI	D3
4–5	Identify API	Fingerprint cascade, block keys, SDK alpha	D1, D2
6–7	Score API	Velocity, scoring engine, SIM swap, quota	D1
8	Mid-term defense	Design review, progress demo, supervisor log	Mid-term report
9–10	Async layer	NATS subjects, Python workers, profiling	D4
11–12	Validation	k6 runs, 168 tests, bug fixes, tuning	D5
13–14	Final defense	Final report, demo, documentation, presentation	D6, D7

9 Bibliography/References

References

- [1] R. J. Bolton and D. J. Hand, Statistical fraud detection: A review, *Statistical Science*, 17(3):235–255, 2002.
- [2] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, Calibrating probability with undersampling for unbalanced classification, *2015 IEEE Symposium Series on Computational Intelligence*, 2015.
- [3] P. Eckersley, How unique is your web browser? *Privacy Enhancing Technologies Symposium (PETS 2010)*, EFF, 2010.
- [4] U. Fiore, A. De Santis, F. Perla, P. Zanetti, and F. Palmieri, GAN-based augmentation for credit-card fraud detection, *Information Sciences*, 479:448–455, 2019.
- [5] Financial Intelligence Unit Nepal, *Annual Report 2023/24*, Nepal Rastra Bank, Kathmandu, 2024.
- [6] P. Laperdrix, W. Rudametkin, and B. Baudry, Beauty and the beast: Diverting modern web browsers to build unique fingerprints, *IEEE Symposium on Security and Privacy*, 2016.
- [7] B. B. Mjaaland, J. Heeger, and S. D. Wolthusen, SIM swap: A new insider attack in the mobile banking ecosystem, *IEEE CSR*, 2021.
- [8] A. Vastel, P. Laperdrix, W. Rudametkin, and R. Rouvoy, FP-STALKER: Tracking browser fingerprint evolutions, *IEEE Symposium on Security and Privacy*, 2018.
- [9] J. West and M. Bhattacharya, Intelligent financial fraud detection: A comprehensive review, *Computers & Security*, 57:47–66, 2016.